

Progetto del software

Prova scritta – 29 giugno 2026 – 2h

PARTE 1 – RISPOSTA SINGOLA - Ogni domanda ha una sola risposta VERA.

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
- Una risposta errata fa perdere il punteggio negativo riportato a fianco della domanda
- Una risposta lasciata in bianco viene valutata 0

1. **(3, -.5)** Il comando di *commit* Git

- a) Ha come effetto la creazione di una “istantanea” del codice, ed il caricamento automatico sul server remoto denominato “origin”
- b) Ha come effetto la creazione di una “istantanea” del codice, ed il caricamento automatico sul server remoto definito come parametro (che può essere anche “origin”, ma non è detto)
- c) Ha come effetto la creazione di una “istantanea” del codice, ed il caricamento automatico sul server remoto denominato “master” o “main” (a seconda della versione di Git)
- d) Nessuna delle precedenti

2. **(3, -.5)** Lo *Unified Modeling Language* (UML)

- a) E' uno standard per la rappresentazione di un progetto software a vari livelli di astrazione
- b) E' uno standard per la rappresentazione di un progetto software che però deve essere implementato col paradigma a microservizi
- c) E' uno standard per la rappresentazione di un progetto software che però deve essere implementato col paradigma ad oggetti
- d) Nessuna delle precedenti

3. **(3, -.5)** Rispetto alla programmazione “a toolkit/libreria”, la programmazione “a framework”

- a) Si applica solo alla programmazione funzionale
- b) Rende il codice più robusto, ma meno scalabile
- c) Prevede la creazione di componenti software che rispettino un'interfaccia definita dal framework stesso, e che li si inietta (*injection*) in esso
- d) Non prevede l'uso di librerie che non siano parte del framework stesso

PARTE 2 – (POSSIBILI) RISPOSTE MULTIPLE -

Ogni domanda può avere da zero a quattro risposte CORRETTE.

- Ogni risposta esatta viene calcolata: +1
 - Ogni risposta errata viene calcolata: -0.5
 - Una risposta lasciata in bianco viene calcolata: 0
4. Una chiamata sincrona
- a) Generalmente prevede una qualche forma di *callback* per gestire il corretto/l'errato funzionamento della chiamata
 - b) Non è possibile in una architettura CLEAN
 - c) Richiede una profonda conoscenza dei meccanismi di *callback*
 - d) Può avvenire in modalità *fire-and-forget*, ed in quel caso non si hanno garanzie dirette della sua corretta presa in carico da parte del server
5. Il principio di *Liskov Substitution*
- a) Non si applica alla programmazione ad oggetti
 - b) Si applica alla programmazione ad oggetti
 - c) Richiede necessariamente la creazione di interfacce (intese come *interfaces* di un linguaggio ad oggetti)
 - d) E' incompatibile col design pattern *Abstrct Factory*
6. Il modello di sviluppo software "a *Waterfall*"
- a) Prevede la suddivisione dello sviluppo del software in diverse fasi
 - b) Non è mai compatibile con lo sviluppo web
 - c) Non copre la fase di "test" di un software
 - d) E' compatibile con CLEAN architecture
7. In una architettura CLEAN
- a) Le dipendenze vanno sempre verso l'"interno", ossia verso le cosiddette *Entity*
 - b) Le dipendenze vanno sempre verso l'"esterno", ad esempio verso le cosiddette *Presentation* o verso i driver dei database
 - c) Le dipendenze possono andare sia verso l'interno che verso l'esterno, a seconda se usiamo *Dependency Injection* o meno
 - d) A lungo termine può rappresentare uno svantaggio per il team di sviluppatori, perché aggiunge complessità inutile
8. Il diagramma UML di sequenza
- a) Consente di modellare l'avanzamento temporale dell'interazione i vari componenti del software
 - b) Consente di modellare il tipo di interazione (es: sincrona/asincrona) fra i vari componenti del software
 - c) Consente di modellare le chiamate a funzione fra i vari componenti, dove previste
 - d) Consente di modellare le chiamate REST fra i vari componenti del software, dove previste

PARTE 3 – DOMANDE APERTE

- Una risposta esatta fa acquisire il punteggio positivo riportato a fianco della domanda
- Una risposta errata può eventualmente causare una penalità che dipende dalla gravità dell'errore
- Una risposta lasciata in bianco viene calcolata: 0
- L'eventuale sfioramento del limite di righe o parole (laddove imposto), porterà a una decurtazione di un punto per ogni riga. Eventuali schematici e listati di codice non verranno presi in considerazione nel calcolo delle righe
- **SI RICORDA CHE L'UNICO FOGLIO DA CONSEGNARE E' IN CALCE AL COMPITO. QUESTO FOGLIO, PUO' SERVIRE ESCLUSIVAMENTE COME "BRUTTA COPIA". EVENTUALI RISPOSTE SCRITTE IN QUESTO FOGLIO NON VERRANNO PRESE IN CONSIDERAZIONE**

9. **(8 pt)** Si descriva il concetto di *code smell* in non più di 2 righe

Nome e Cognome _____ Matricola: _____

10. **(5 pt)** Quali sono i vantaggi e gli svantaggi di un'architettura client/server?

Nome e Cognome _____ Matricola: _____

Progetto del software

Prova scritta – 29 giugno 2026 – 2h

Indicare le risposte corrette apponendo una croce nella casella corrispondente. Per superare la prova bisogna aver raggiunto almeno 9 punti nelle domande a risposta singola/multipla, ed almeno 15 complessivamente. Questa è l'unica pagina che dovete consegnare.

	Risposte				Punti/ Penalità	
	A	B	C	D		
1					3	-0.5
2					3	-0.5
3					3	-0.5
4						
5						
6						
7						
8						

Risposta alla domanda 9 (8 pt):

Nome e Cognome _____ Matricola: _____

Risposta alla domanda 10 (5 pt):